

Observability and auditability: Ask yourself a simple question: can you reconstruct what your agent did and why? Not just the final output. The full decision chain — which tools were called, in what order, with what inputs, and what intermediate reasoning produced each step. In regulated industries, this isn't optional. It's a prerequisite for any human review process, and increasingly for regulatory defensibility too.

TRIGGER / USER REQUEST

natural language intent or system event

ORCHESTRATOR LAYER

task decomposition · agent routing · step sequencing
[trace_id issued]

AGENT EXECUTION LAYER

tool calls · LLM inference · subagent spawning [span per tool call logged]

TOOL / INTEGRATION LAYER

APIs · databases · external services · message brokers

EMITTED → Distributed trace · Decision log · Cost meter · Audit record

Table 1: Agentic observability stack

The key architectural requirement here is a persistent trace identifier that propagates through every layer: orchestrator, agent, tool call, and external system. Any incident should be reconstructable from a single ID. Without it, post-hoc debugging of a multi-agent workflow is effectively forensic archaeology. And nobody wants to do that at 3 a.m.

Failure modes and recovery: I often describe agentic failure modes as ‘creative failures’ — and not in a good way. A tool call that returns a plausible but incorrect result won't throw an exception. The agent will use it. A reasoning loop that partially executes a multi-step task and then crashes leaves the world in a state that's neither committed nor rolled back. It's a category of failure that conventional error handling simply wasn't designed for. Recovery patterns must be built in deliberately, not bolted on later.

State management and idempotency: You need durable state stores rather than in-memory context, explicit checkpoint markers, and idempotent tool implementations so that a resumed workflow doesn't duplicate side effects. Think about what happens when an agent resends an invoice because it had no durable record of having already succeeded. That's not hypothetical. That's exactly what happens when teams treat idempotency as a nice-to-have rather than a correctness requirement.

Pillar two

Security and trust

Here's a framing that I find useful: agents are principals. They act in the world. They hold credentials, make decisions, and invoke systems on behalf of users and organisations. That means the security model that governs them can't be retrofitted from user-centric IAM. It must be designed from the ground up for the specific properties of autonomous action.

Access control and least-privilege for agents: The most common mistake I see in early agentic deployments is giving agents the same permissions as the humans they're acting for. An agent that can read everything a user can access and write everywhere a user can modify has an attack surface proportional to the entire user base. The blast radius of a prompt injection or reasoning failure is unbounded. And this is not a theoretical risk. OWASP's 2025 Top 10 for LLM Applications ranks prompt injection the #1 critical vulnerability, present in over 73% of production AI deployments assessed in security audits. In late 2025, researchers demonstrated how a malicious instruction hidden in an email caused an AI agent to compose and send a resignation letter on behalf of the user, instead of the out-of-office reply it was asked to write. OpenAI, responding to the finding, acknowledged that prompt injection "may never be fully solved." That's not a reassuring statement when your agent has written access to your production systems.

FAILURE TYPE	WHAT HAPPENS	MITIGATION PATTERN
Reasoning drift	Goal substitution: agent optimises for proxy metric, not original intent	Anchored system prompts + HITL approval gates
Partial execution	System in inconsistent state; neither committed nor rolled back	Saga/compensating tx pattern; durable workflow checkpointing
Tool hallucination	Downstream logic built on fabricated data; silent corruption	Tool result schema validation; confidence thresholds + fallback
Cascading tool calls	Quota exhaustion, rate limits hit, runaway cost accumulation	Max-step limits; circuit breakers; per-workflow budget caps

Table 2: Agentic failure taxonomy and mitigations